

Web Technology

Unit- IV

Topic:

JSP, Web Services, Struts

Syllabus

JSP:

- JSP: Introduction to Java Server Pages,
- JSP and Servlets,
- running JSP applications,
- Basic JSP,
- JavaBeans classes and JSP,
- Support for the Model-View-Controller paradigm,
- JSP related technologies

JSP

What is JSP

- JSP technology is used to create web application just like Servlet technology.
- It can be thought of as an extension to servlet because it provides more functionality than servlet such as expression language, jstl etc.
- A JSP page consists of HTML tags and JSP tags.
- The jsp pages are easier to maintain than servlet because we can separate designing and development.
- It provides some additional features such as Expression Language, Custom Tag etc.

JSP Vs Servlet

JSP	Servlet
JSP is Scripting Language that generates dynamic content.	Servlets are the JAVA Program that can be compiled to generate dynamic Content
JSP Runs Slower than Servlet	Servet runs faster than JSP
In Model-View- Controller JSP act as view	In Model-View- Controller Servlet act as Controller
JSP Can build custom tags	Servlets can not build custom tags
It is Easier to Code in JSP	The Servlets are basically Complex JAvA Programs

Advantage of JSP over Servlet

Extension to Servlet

Easy to maintain

Less code than Servlet

Fast Development: No need to recompile and
redploy

Advantage of JSP over Servlet

1) Extension to Servlet

- JSP technology is the extension to servlet technology.
- We can use all the features of servlet in JSP.
- In addition to, we can use
 - implicit objects,
 - predefined tags,
 - expression language and
 - Custom tags in JSP,
 - that makes JSP development easy.

Advantage of JSP over Servlet

2) Easy to maintain

- JSP can be easily managed because we can easily separate our business logic with presentation logic.
- In servlet technology, we mix our business logic with the presentation logic.

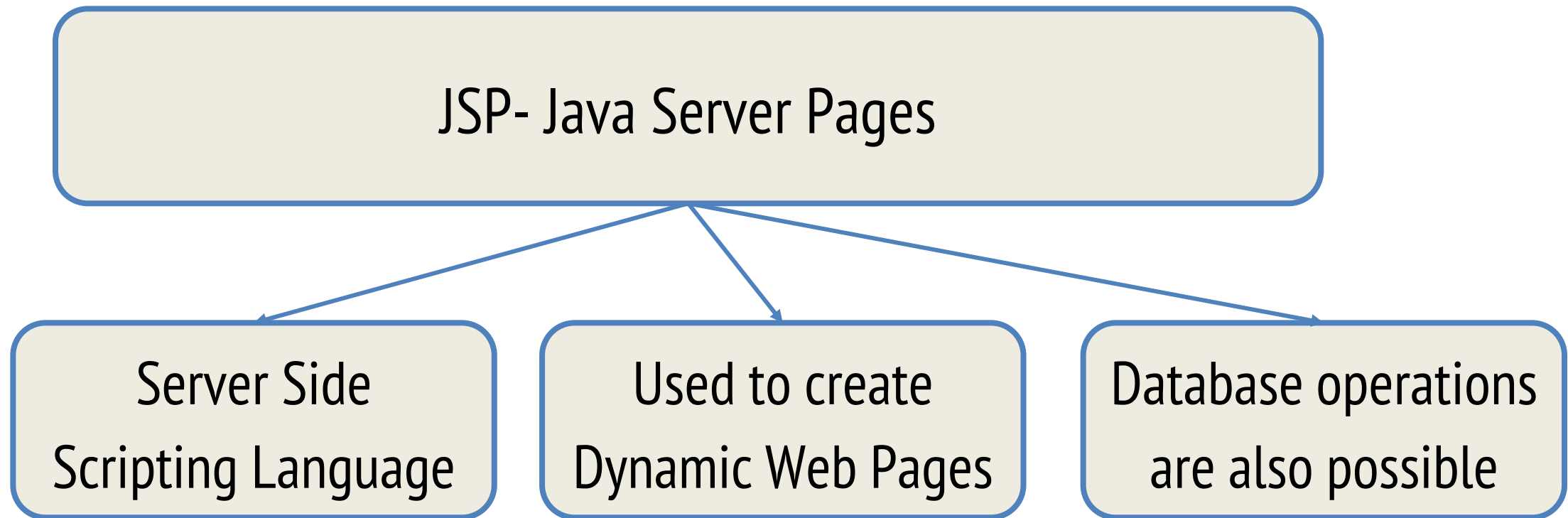
3) Fast Development: No need to recompile and redeploy

- If JSP page is modified, we don't need to recompile and redeploy the project.
- The servlet code needs to be updated and recompiled if we have to change the look and feel of the application.

4) Less code than Servlet

- In JSP, we can use a lot of tags such as action tags, jstl, custom tags etc. that reduces the code.

What is JSP



Creating a JSP Page

A JSP page looks similar to an HTML page, but a JSP page also has Java code in it.

```
graph TD; A["A JSP page looks similar to an HTML page, but a JSP page also has Java code in it."] --> B["start with <%"]; A --> C["and ends with %>."];
```

start with <%

and ends with %>.

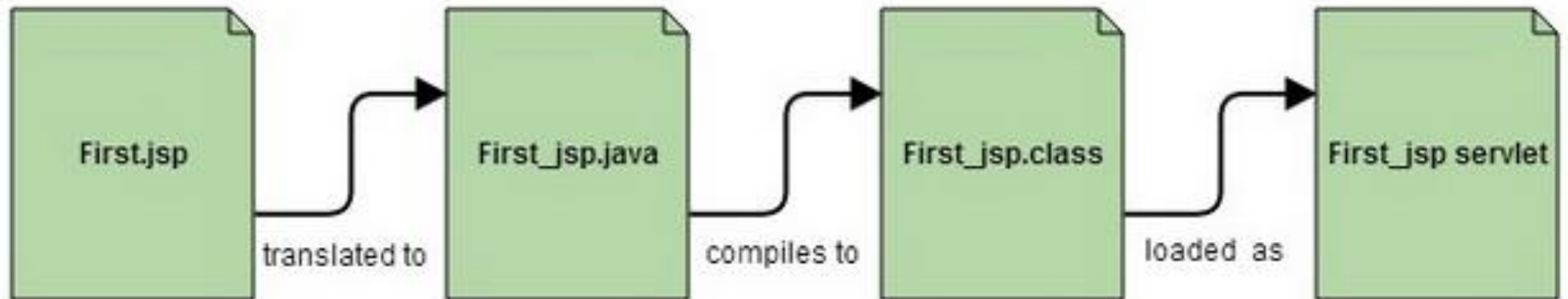
JSP Life Cycle

Life cycle of a JSP Page

The JSP pages follows these phases:

1. Translation of JSP Page
2. Compilation of JSP Page
3. Class loading (class file is loaded by the class loader)
4. Instantiation (Object of the Generated Servlet is created).
5. Initialization (`jspInit()` method is invoked by the container).
6. Request processing (`_jspService()` method is invoked by the container).
7. Destroy (`jspDestroy()` method is invoked by the container).

Life cycle of a JSP Page



JSP Compilation

- When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page.
- If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.
- The compilation process involves three steps –
 - Parsing the JSP. (Usually the `parse()` method receives some string as input, "extracts" the necessary information from it and converts it into an object of the calling class. For example, it received a string and returned the date that was "hiding" in this string.
)
 - Turning the JSP into a servlet.
 - Compiling the servlet.

JSP Initialization

- When a container loads a JSP it invokes the `jspinit()` method before servicing any requests.
- If you need to perform JSP-specific initialization, override the `jspinit()` method –

```
public void jspinit(){  
    // Initialization code...  
}
```

- Typically, initialization is performed only once and as with the servlet `init` method, you generally initialize database connections, open files, and create lookup tables in the `jspinit` method.

JSP Execution

- This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.
- Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the `_jspService()` method in the JSP.
- The `_jspService()` method as follows –

```
void _jspService(HttpServletRequest request, HttpServletResponse response)
{
    // Service handling code...
}
```
- This is responsible for generating the response for that request of the HTTP methods, i.e, GET, POST, DELETE, etc.

JSP Cleanup

- The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container.
- The `jspDestroy()` method is the JSP equivalent of the `destroy` method for servlets.
- The `jspDestroy()` method has the following form –

```
public void jspDestroy() {  
    // Your cleanup code goes here.  
}
```

JSP in Detail

Elements of JSP

Scripting Element	Use	Syntax
Comment	Used to add comments	<code><%-- comment --%></code>
Directive	Directives controls the processing of entire JSP Page.It gives direction to the server regarding the processing of a page.	<code><%@ directive %></code>
Declaration	Used to declare variable or method in JSP	<code><%! declarations %></code>
Scriptlet	Java code is written inside Scriptlet tag.	<code><% scriptlet %></code>
Expression	Used to represent the expression in JSP page.	<code><%= expression %></code>

Elements of JSP: Examples

JSP Comments

`<%-- This is JSP comment --%>`

The Scriptlet

```
<%  
    out.println("Welcome");  
%>
```

JSP Declarations

```
<%! int i = 0; %>  
<%! int a, b, c; %>
```

Examples

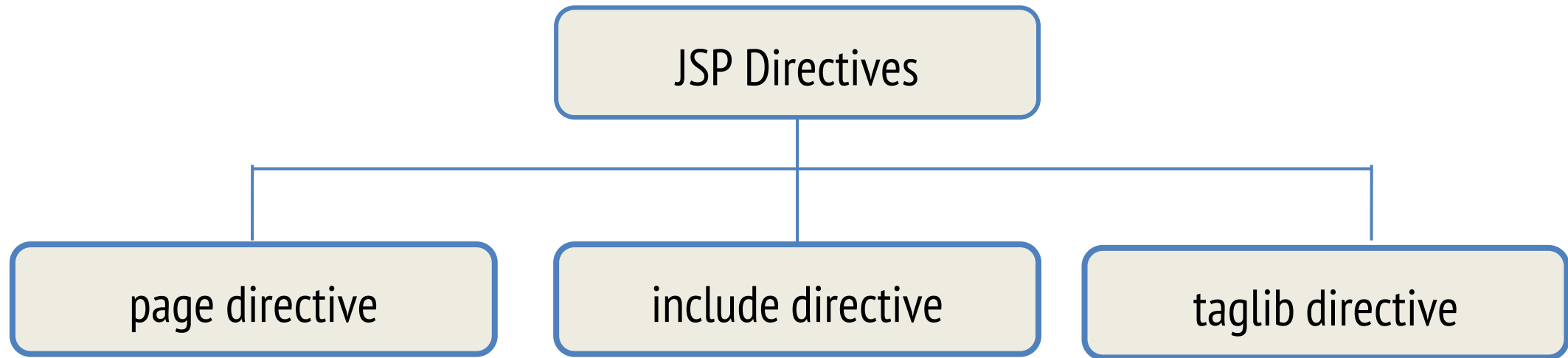
JSP Expression

`<%= "welcome to jsp" %>`

JSP Directives

`<%@ directive attribute="value" %>`

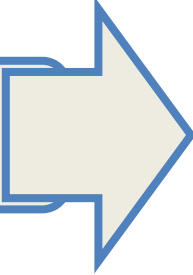
JSP Directives – It control the processing of entire JSP page. It gives direction to the server regarding the processing of a page



Syntax of Directive: `<%@ directive name attribute name="Value">`

JSP Directives

page directive



Used to provide information about the page.

Various Attributes of Page Directives are:

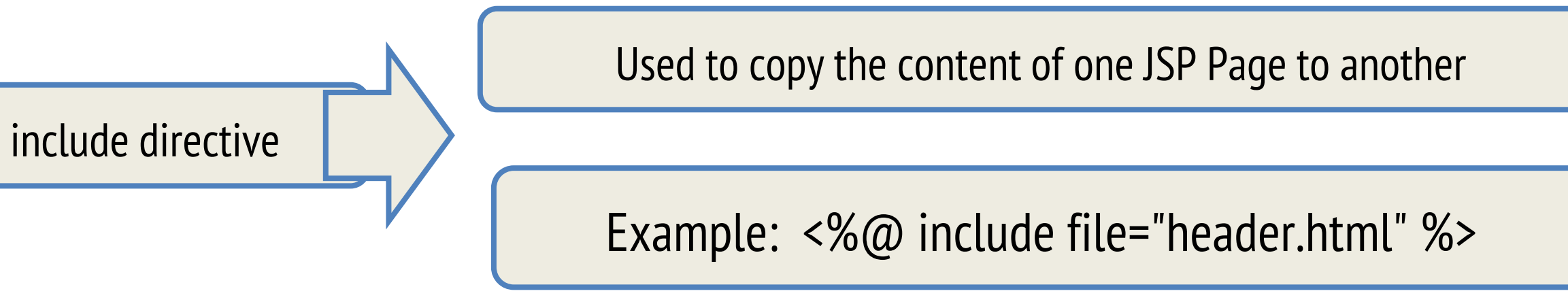
- | | | | |
|------------|-------------|----------------|--------------|
| 1. import | 2. language | 3. ContentType | 4. extends |
| 5. session | 6. buffer | 7. info | 8. errorPage |

Example:

1. `<%@ page import="java.io.*" %>`
2. `<%@ page language="java" contentType="text/html" %>`

JSP Directives

include directive



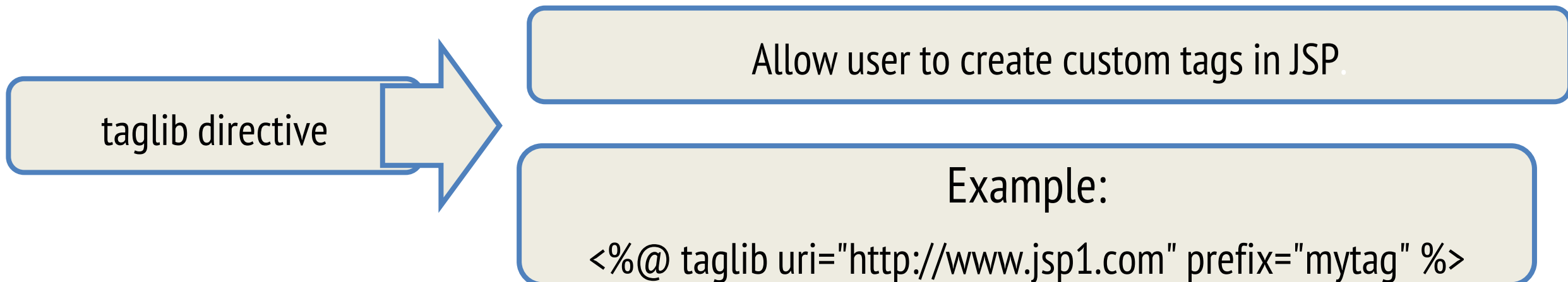
```
graph LR; A[include directive] --> B[Used to copy the content of one JSP Page to another]; A --> C[Example: <%@ include file="header.html" %>];
```

Used to copy the content of one JSP Page to another

Example: `<%@ include file="header.html" %>`

JSP Directives

taglib directive



Allow user to create custom tags in JSP.

Example:

```
<%@ taglib uri="http://www.jsp1.com" prefix="mytag" %>
```

JSP Basic Examples

Scripting Elements :

1. Expression – the expression tag is used to represent expression in JSP page
syntax - `<%=java Expression %>`

```
<html>
<head>
<title> JSP Expression Demo</title>
</head>
<body>
Value of Expression is :
<%= (10*20)%>
</body>
</html>
o/p – value of expression is : 200
```

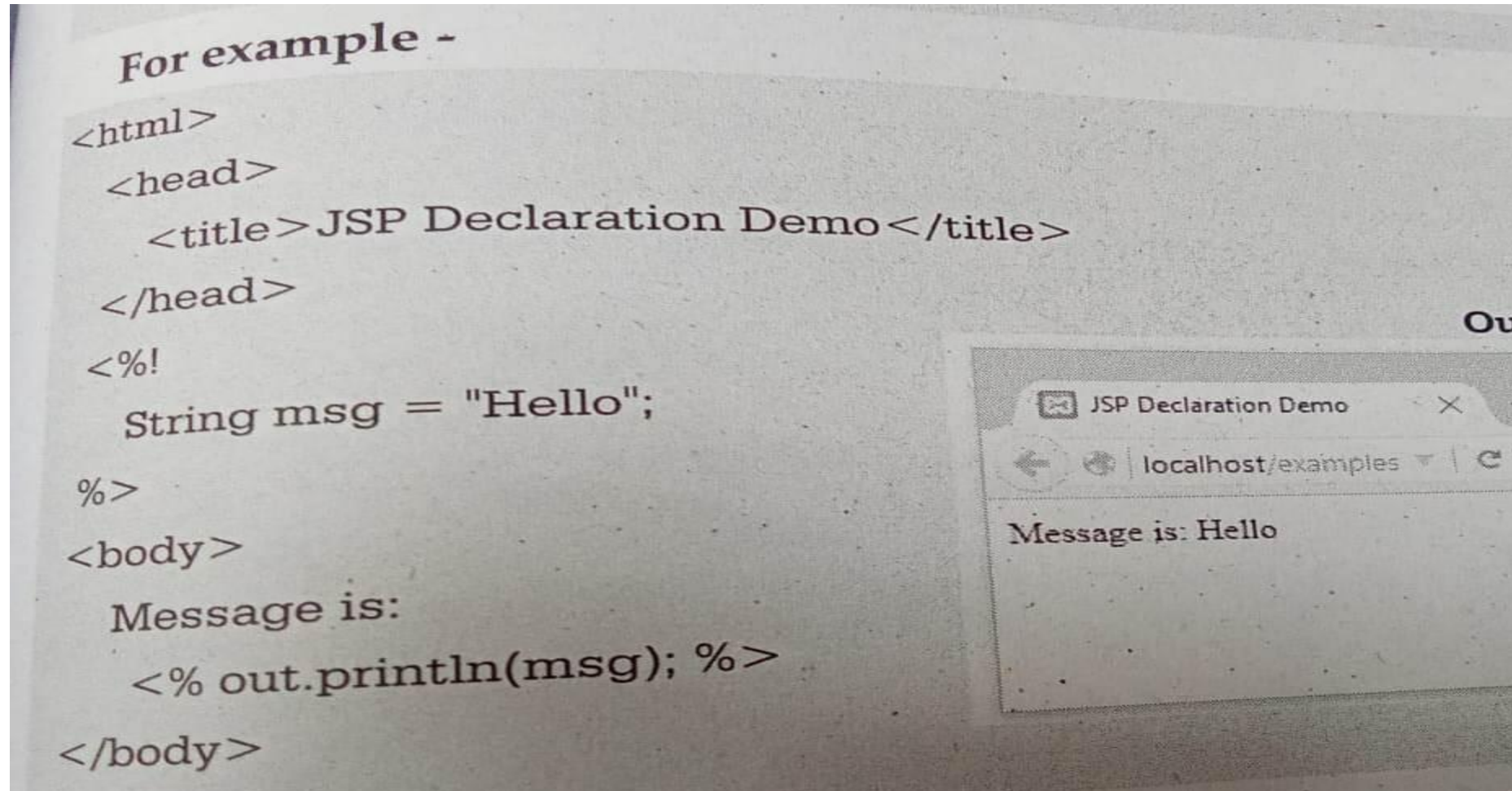
2. Scriptlet - the code that appears between the <% and %> delimiters is called a scriptlet. it is a java code enclosed within <% and %> tags.

Templatetext.jsp

```
<%@ page language="java" contentType="text/html" %>
<html>
  <head>
    <title>Demo for Tempplate Text</title>
  </head>
  <body bgcolor="gray">
    <h1>Twinkle Twinkle little stars</h1>
    <h2>How I wonder what you are!!!</h2>
    <li>like a diamond in the sky</li>
    <p>
      <% out.print("JSP is eqaul to HTML and JAVA "); %>
    </p>
  </body>
</html>
```

3. Declaration – we can declare a variable or method in JSP inside declaration tag. We can declare static member, instance variable and methods inside declaration tag –

`<%! Declaration code %>`



JSP Code- To display even numbers from 1 to 50

```
<%@ page language="java" contentType="text/html" %>
```

```
<%
```

```
    for ( i=1; i<=50; i++)
```

```
    {
```

```
        if( i%2 == 0)
```

```
            out.println("No is Even"+ i);
```

```
    }
```

```
%>
```

jsp:include

- Using the `<jsp:include>` action we can include different files in current jsp pages

jsp:include- example

- In this example, index.jsp file includes the content of the printdate.jsp file.

File: index.jsp

```
<h2>this is index page</h2>  
<jsp:include page="printdate.jsp" />  
<h2>end section of index page</h2>
```

- **File: printdate.jsp**

```
<% out.print("Good Morning"); %>
```


jsp:forward action tag

- The jsp:forward action tag is used to forward the request to another resource it may be jsp, html or another resource.
- Here is a simple demonstration in which we have created two web pages the first one is main JSP page named as first.jsp in which we are using JSP action forward . And forwarding current request to an HTML page which is named as second.html

jsp:forward Example

first.jsp

```
<h2>this is index page</h2>  
<jsp:forward page="second.html" />
```

Second.html

```
<body>  
<strong>Good Morning</strong>  
</body>
```

For getting output open web page and open main jsp page that is first.jsp and you will get following output

jsp:useBean (Java Beans)

- It is a reusable software component. We can use simple java bean in the JSP.
- Beans are used in JSP pages as the instance of class. We must specify the scope of bean in JSP page. Here the scope of bean means the range time span of the bean for its existence in JSP.
- Moreover, it provides the easy maintenance.
- Bean is basically a java class with get and set methods

Example

Q.1 Write a JSP document to display number of times user is visiting particular page

include Action Vs include Directive

include Directive	include Action
Includes the file at transition time.	Includes file at run time
If include file is changes but not JSP, then changes will not be reflected	If include file is changes but not JSP, then changes will be reflected
Syntax: <%@ include file="filename" %>	Syntax: <jsp:include page="Filename" />
We cannot pass parameter to the included file.	We can pass parameter to the included file using param action.

JSP implicit Objects

JSP implicit Objects

- Implicit objects are predefined variables used to access request & application data.
- There are 9 jsp implicit objects.
- These objects are created by the web container that are available to all the jsp pages.

Object	Type	Examples
out	JspWriter	<code>PrintWriter out=response.getWriter();</code>
request	HttpServletRequest	<code>request.getParameter("t1");</code>
response	HttpServletResponse	<code>response.sendRedirect("http://www.google.com");</code>
config	ServletConfig	<code>String driver=config.getInitParameter("dname");</code>
application	ServletContext	<code>String driver=application.getInitParameter("dname");</code>
session	HttpSession	<code>session.setAttribute("user",name);</code>
pageContext	PageContext	<code>pageContext.setAttribute("user",name,PageContext.SESSION_SCOPE);</code>
page	Object	<code>Object page=this;</code>
exception	Throwable	<code><%= exception %></code>

Java Beans classes and JSP

- It is a reusable software component. We can use simple java bean in the JSP.
- Beans are used in JSP pages as the instance of class. We must specify the scope of bean in JSP page. Here the scope of bean means the range time span of the bean for its existence in JSP
- Bean is basically Java class with getxx and setxx methods.

Characteristics for JavaBean class

- Java bean class contains a default no argument constructor. This constructor must have the access specifier as public.
- It should be serializable and should implement serializable interface.
- It can have any number of properties which can be read or written .
For these properties we write getter and setter methods.

JavaBean Properties

A JavaBean property is a named attribute that can be accessed using the object

The JavaBean properties can be accessed through two methods namely

1. **getPropertyName()** – This method is called accessor (In Java accessors are used to get the value of a private field and mutators are used to set the value of a private field. Accessors are also known as getters and mutators are also known as setters) method. It returns the property value. This method is written in some special way. For example – if the property is rollNo, then the method name for rollNo is **getRollNo()**
 2. **setPropertyName()** – this is called mutator method. This method is generally used to assign the value to the properties. This method is written in some special way. For example – if the property is rollNo then method name will be **setRollNo()**
- A read-only attribute will have only a **getPropertyName()** method, and a write-only attribute will have only a **setPropertyName()** method.

Example – Bean program using JSP for Employee Information

- Step 1 : Create a Bean program in some package. We have create a folder (as a pckage)named BeanProg. The code for JavaBean class is written as follows –

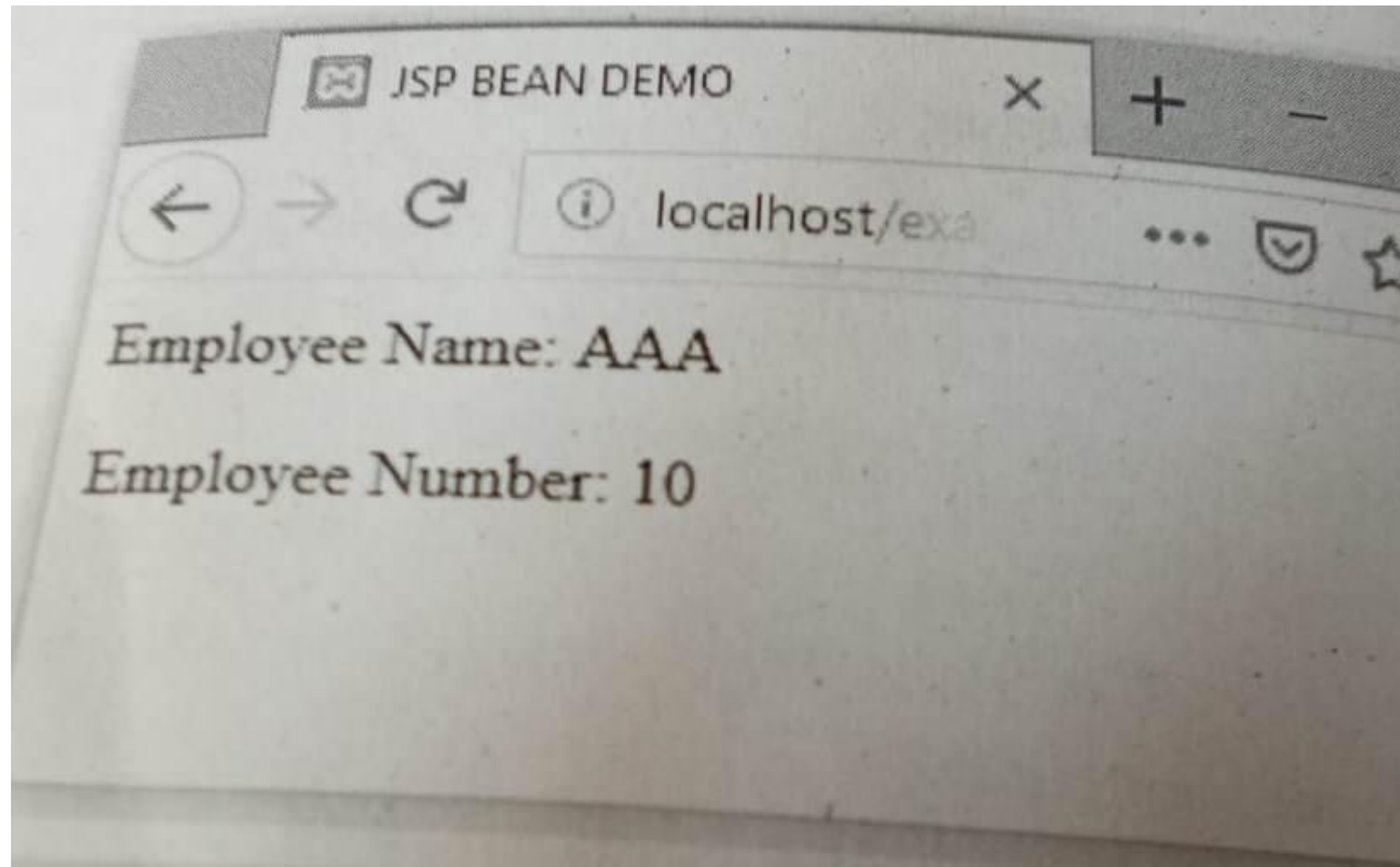
```
package BeanProg;

    public class StudentsBean implements java.io.Serializable {
    private String empName = null;
    private int empNo = 0;
    public Employee() { // no argument constructor
    }
    public String getempName(){
    return empName;
    }
    public int getEmpNo() {
    return empNo; }
    public void setEmpName(String empName){
    this.empName = empName; }
    public void setEmpno(int empNo) {
    This.empNo = empNo;
    }
    }
```

Step 2 – The JSP code can be written as follows -

- `<html>`
- `<head>`
- `<title>JSP BEAN DEMO</title>`
- `</head>`
- `<body>`
- `<jsp:useBean id = "empID" class = "BeanProg.Employee">`
- `<jsp:setProperty nae = "empID" property = "empName" value = "AAA"/>`
- `<jsp:setProperty name = "empID" property = "empNo" value = '10'/>`
- `</jsp:useBean>`
- `<p>Employee Name;`
- `<jsp:getproperty name = "empID" property = "empName"/>`
- `</p>`
- `<p>Employee Number;`
- `<jsp : getProperty name = "empID" property = "empNo"/>`
- `</p>`
- `</body>`
- `</html>`

Step 3 – Start tomcat and open JSP page in web browser you will get the output as follows -



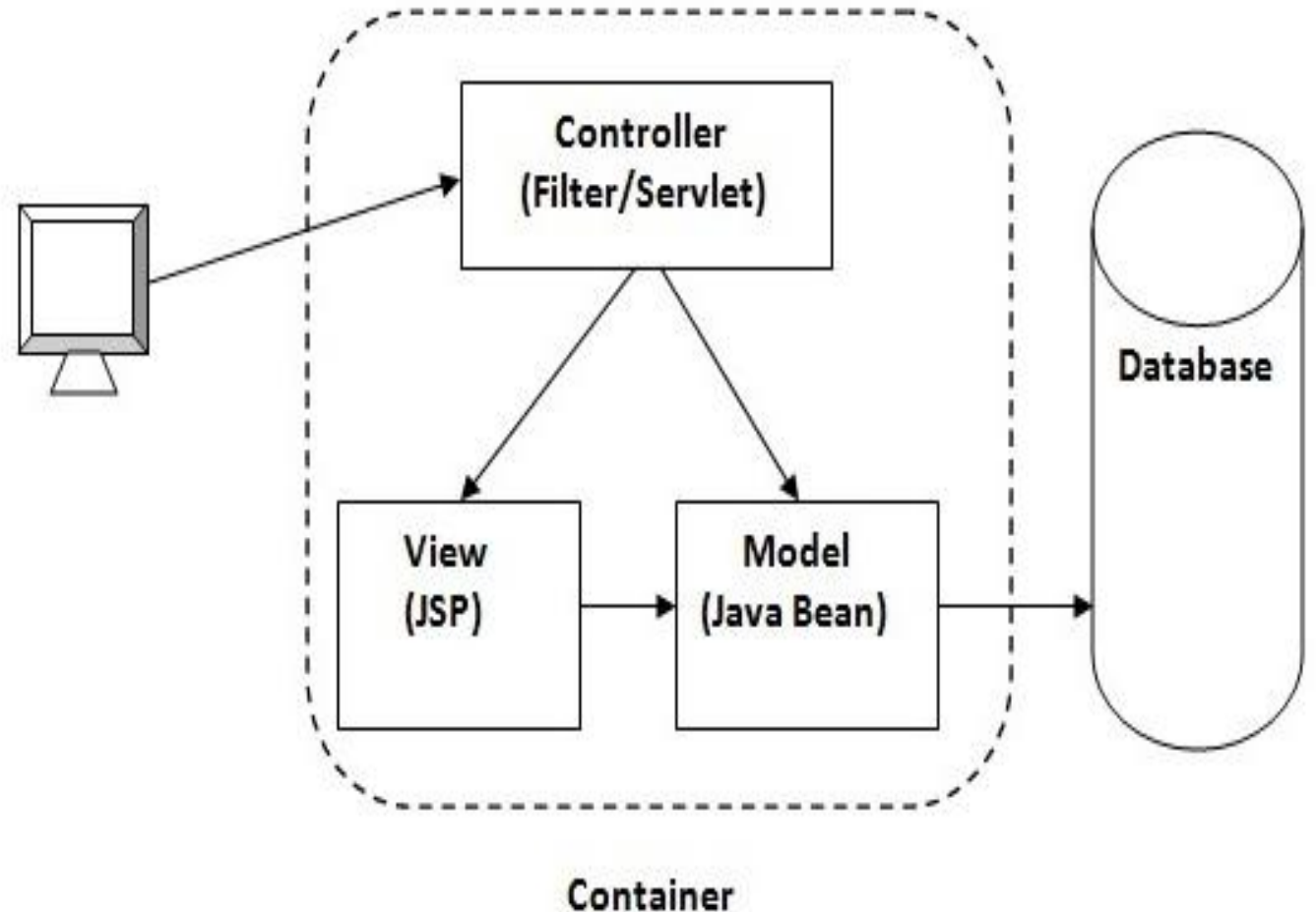
JSP - Support to Model-View Controller Paradigm

JSP - Support to Model-View Controller Paradigm

- MVC stands for Model View and Controller. It is a design pattern that separates the business logic, presentation logic and data.
- Controller acts as an interface between View and Model. Controller intercepts all the incoming requests.
- Model represents the state of the application i.e. data. It can also have business logic.
- The business logic means coding logic applied for manipulation of application data. The presentation refers to the code written for look and feel of the web page.
- View represents the presentaion i.e. UI(User Interface).

JSP - Support to Model-View Controller Paradigm

According to MVC design model , the model corresponds to business logic, view corresponds to presentation and controller corresponds to request processing



Advantages of using MVC design model

1. The use of MVC architecture allows the developer to keep the separation between business logic, presentation and request processing.
2. Due to separation it becomes easy to make changes in presentation without disturbing the business logic.
3. The changes in presentation are often required for accommodating the new presentation interfaces

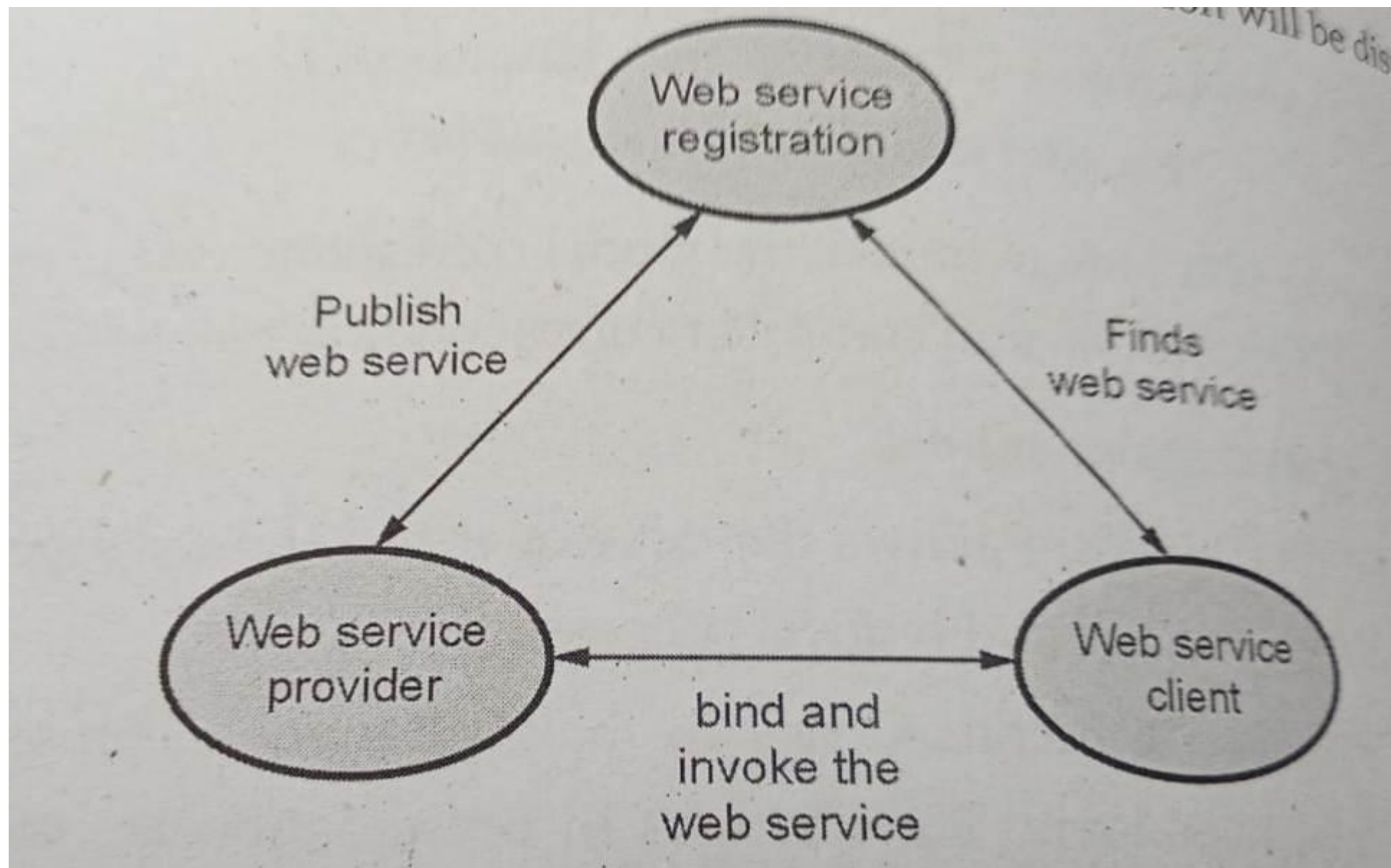
Web Services Concept

1. The web services are the software system that are displayed by the web browser using the web protocol.
2. These software systems are used by the some software applications rather than by end-users directly.
3. Web service is a software system designed which is independent of specific hardware or software on which it is running

Web Services Concept

Examples of web service are -

1. credit card validation system – for using this service the client simply enters the ID and password for the credit card and web service reports the validity of it.
2. Whether forecast system - the clients submits the location and web service returns the information about current whether as well as it may report some predictions
3. Currency converter – the client enter the source and destination currency along with the amount and appropriate converted information will be displayed.

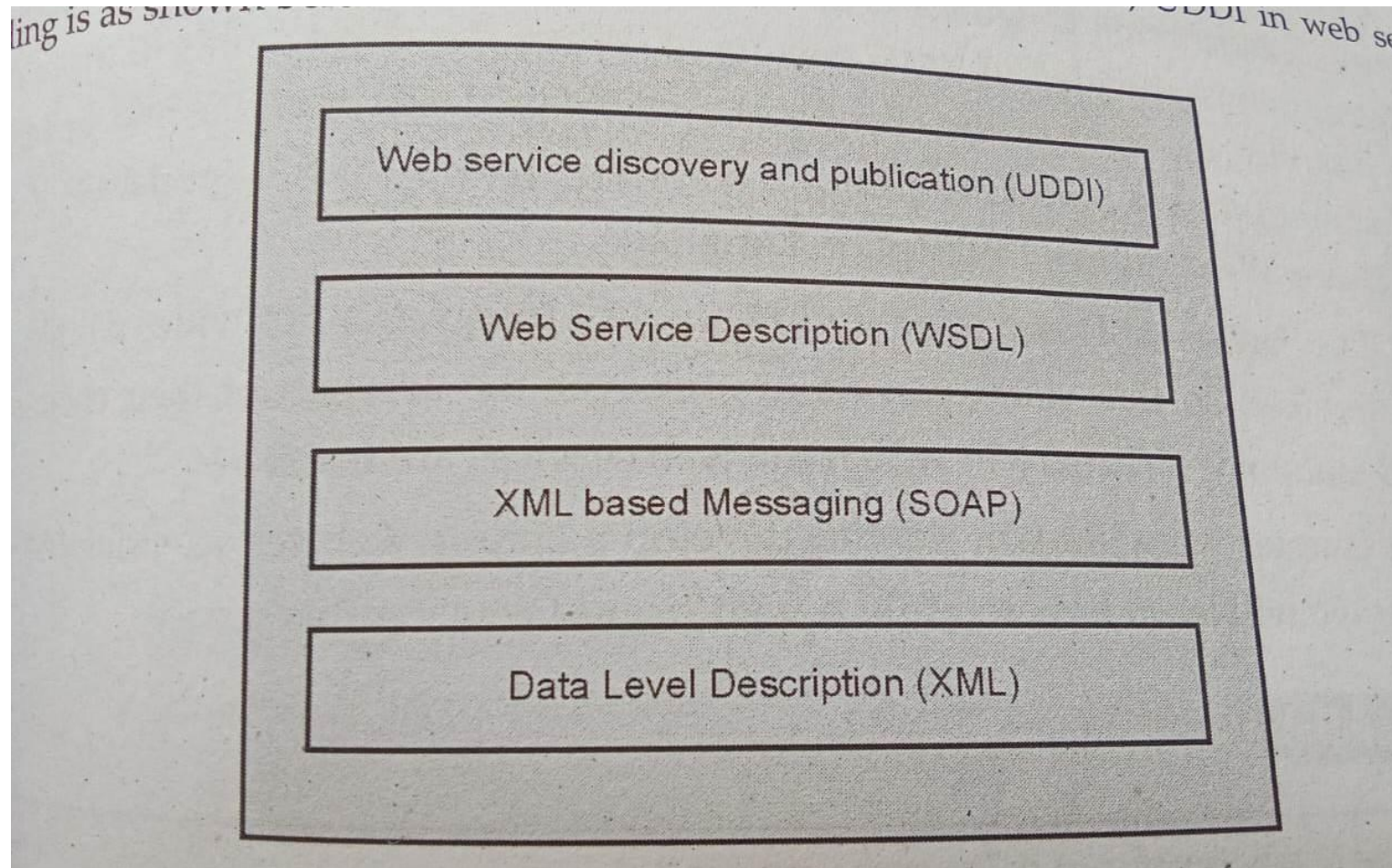


- Following are the steps that a web service model follows-
- step 1 – In first step, a service provider publishes a web service in the web service registry.\
- step 2 – A web client who demands for some web service searches in the registry. After finding a match for the desired web service in the registry, the client choose it.
- Step 3 – The client binds to the corresponding web service provider and invokes for the service

Components of web services

The role of various components such as XML, SOAP, WSDL, UDDI in web service building is as shown below-1. At the lowest level of web services there are simply XML files that are for data level description.

Components of web services



(A Remote Procedure Call (RPC) is a software communication protocol that one program uses to request a service from another program located on a different computer and network, without having to understand the network's details.)

- At the lowest level of web services there are simply XML files that are for data level description.
- SOAP - Simple Object Access Protocol is a simple XML based protocol which allows exchange of messaging over HTTP.
- Typically web services make use of this protocol for exchange of information .
 - The SOAP makes use of JAX-RPC component. These are JAVA API for XML that makes use of Remote Procedure Calls.
 - Basically SOAP defines message structure, encoding rules and conventions for exchange of information in RPC programming model.
 - The following table describes core JAX-RPC interfaces and classes.

Interface / Class	Description
Service	Main client interface. Used for both static and dynamic invocations.
Service Factory	This is a factory class which is used to create services
Stub	It is used by client for invoking the web services.
Call	For invoking the web services the call is used.
JAXRPCException	This exception is thrown when an error occurs while invoking web service.

- At the higher level very important element of web service lies i.e. WSDL. It is used for abstract description of web services.
- This description is provided in XML document
- When the client wants to make use of web service, the users of the client make read this description file and understand what input data is needed to perform the web service. Such a special file is called WSDL.

- UDDI i.e Universal Description and Discovery Integration, provides a directory of web services , so that client can easily discover the services of their choices. The UDDI maintains a registry in which the web services are registered

Describing web services : WSDL

- The WSDL is a web service descriptor language which is based on XML
- The purpose of this file is to describe the web service and the method of accessing it.
- WSDL is a W3C recommendation
- It contains various definitions . Following elements are described in WSDL file

Elements	Description
types	It specifies the data types of the symbols used by the web service.
messages	It specifies the messages used by the web service.
portType	It specifies the name of the operation (function) defined by the web service
binding	It specifies the name of the protocol of the web service, typically it is SOAP.

The main structure of a WSDL document looks like this:

- `<definitions>`
 - `<types>`
data type definitions.....
`</types>`
 - `<message>`
definition of the data being communicated....
`</message>`
 - `<portType>`
set of operations.....
`</portType>`
 - `<binding>`
protocol and data format specification....
`</binding>`
 - `</definitions>`

Communicating Object Data :SOAP

- Simple object access protocol (SOAP) is a protocol based on XML . It is used by the web services for exchange of information . It is a W3C recommendation.
- This exchange of information is done over HTTP. This is a platform and language independent protocol.
- The client – server communication is based on RPC. The HTTP does not designed to handle the distributed objects that are required by RPC. Hence another application protocol is build over HTTP which popularly known as SOAP

Structure of SOAP

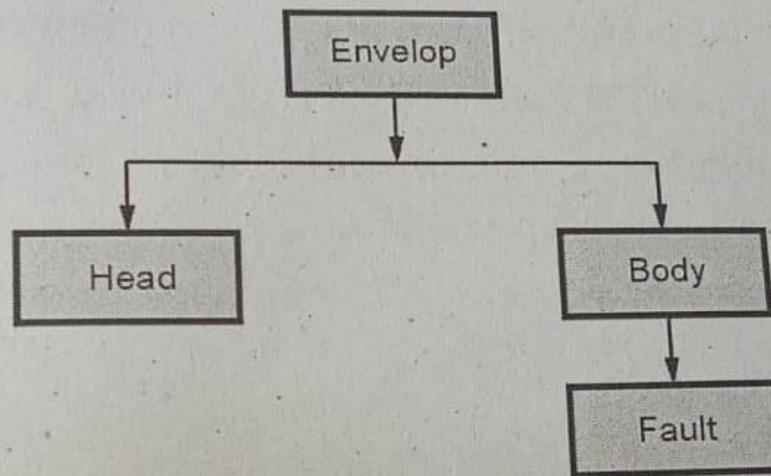


Fig. 4.10.1 SOAP building blocks

1) Envelop

The Envelop is the root of any SOAP document. It identifies that the XML document is a SOAP message. It defines two things - **Namespace** and **encodingStyle**. The Namespace value can be given as -

- All the elements is in the default namespace for the SOAP envelope:
 - <http://www.w3.org/2003/05/soap-envelope>
 - and the default namespace for SOAP encoding and data types is:
 - <http://www.w3.org/2003/05/soap-encoding>
- Envelop – it is the root of any SOAP document. It defines that the XML document as a SOAP message.
- Envelope: This specifies that the XML message is a SOAP message.
- A SOAP message is an XML document containing a header and a body, both encapsulated within the envelope. Any fault is included within the body of the message.
- Header: This part is optional. When present, it can provide crucial information about the applications.
- Body: This contains the actual message being transmitted. Faults are contained within the body tags.
- Fault: This optional section contains the status of the application and any errors. It should not appear more than once in a SOAP message

Skeleton SOAP Message

- `<?xml version="1.0"?>`

```
<soap:Envelope  
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"  
  soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
```

```
<soap:Header>
```

```
...
```

```
</soap:Header>
```

```
<soap:Body>
```

```
...
```

```
<soap:Fault>
```

```
...
```

```
</soap:Fault>
```

```
</soap:Body>
```

```
</soap:Envelope>
```

- The SOAP Envelope Element - The required SOAP Envelope element is the root element of a SOAP message. This element defines the XML document as a SOAP message.

- The xmlns:soap Namespace -
 - Notice the xmlns:soap namespace in the example above. It should always have the value of: "http://www.w3.org/2003/05/soap-envelope".
 - The namespace defines the Envelope as a SOAP Envelope.
 - If a different namespace is used, the application generates an error and discards the message.
- The encodingStyle Attribute
 - The encodingStyle attribute is used to define the data types used in the document. This attribute may appear on any SOAP element, and applies to the element's contents and all child elements.
 - A SOAP message has no default encoding.
 - soap:encodingStyle="URI"

Struts

Outline

Struts: Overview,

architecture,

configuration,

actions,

interceptors,

result types,

validations,

localization,

exception handling,

annotations.

Struts Overview

- Apache Struts 2 is an elegant, extensible framework for creating enterprise-ready Java web applications.
- This framework is designed to streamline the full development cycle from building, to deploying and maintaining applications over time.
- It is an open source product
- **Struts 2 Features**
 - Configurable MVC components
 - POJO based actions
 - AJAX support
 - Integration support
 - Various Result Types
 - Various Tag support
 - Theme and Template support

Struts Overview :Features

1) Configurable MVC components

In struts 2 framework, we provide all the components (view components and action) information in struts.xml file. If we need to change any information, we can simply change it in the xml file.

2) POJO based actions

In struts 2, action class is POJO (Plain Old Java Object) i.e. a simple java class. Here, you are not forced to implement any interface or inherit any class.

3) AJAX support

Struts 2 provides support to ajax technology. It is used to make asynchronous request i.e. it doesn't block the user. It sends only required field data to the server side not all. So it makes the performance fast.

Struts Overview :Features

4) Integration Support

We can simply integrate the struts 2 application with hibernate, spring, tiles etc. frameworks.

5) Various Result Types

We can use JSP, freemarker, velocity etc. technologies as the result in struts 2.

6) Various Tag support

Struts 2 provides various types of tags such as UI tags, Data tags, control tags etc to ease the development of struts 2 application.

7) Theme and Template support

Struts 2 provides three types of theme support: xhtml, simple and css_xhtml. The xhtml is default theme of struts 2. Themes and templates can be used for common look and feel.

Outline

Struts: Overview,

architecture,

configuration,

actions,

interceptors,

result types,

validations,

localization,

exception handling,

annotations.

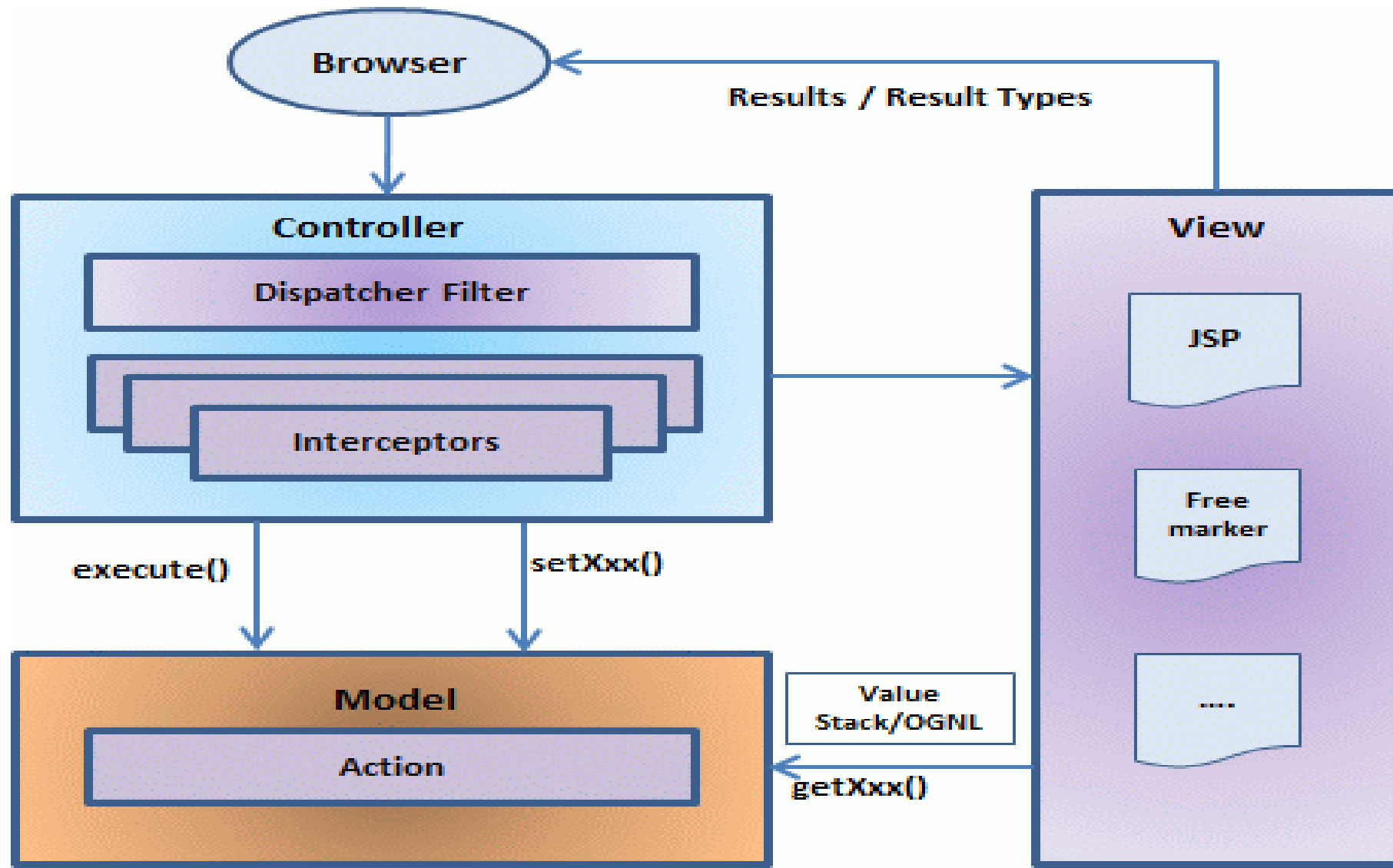
Struts Architecture

- The **struts 2 framework** is used to develop **MVC-based web application**.
- **Model View Controller or MVC** is made up of the following three parts –
 - **Model** The model represents the state (data) and business logic of the application.
 - **View** The view module is responsible to display data i.e. it represents the presentation.
 - **Controller** The controller module acts as an interface between view and model. It intercepts all the requests i.e. receives input and commands to Model / View to change accordingly.
- The Struts framework provides the configurable MVC support. In struts 2, we define all the action classes and view components in struts.xml file.

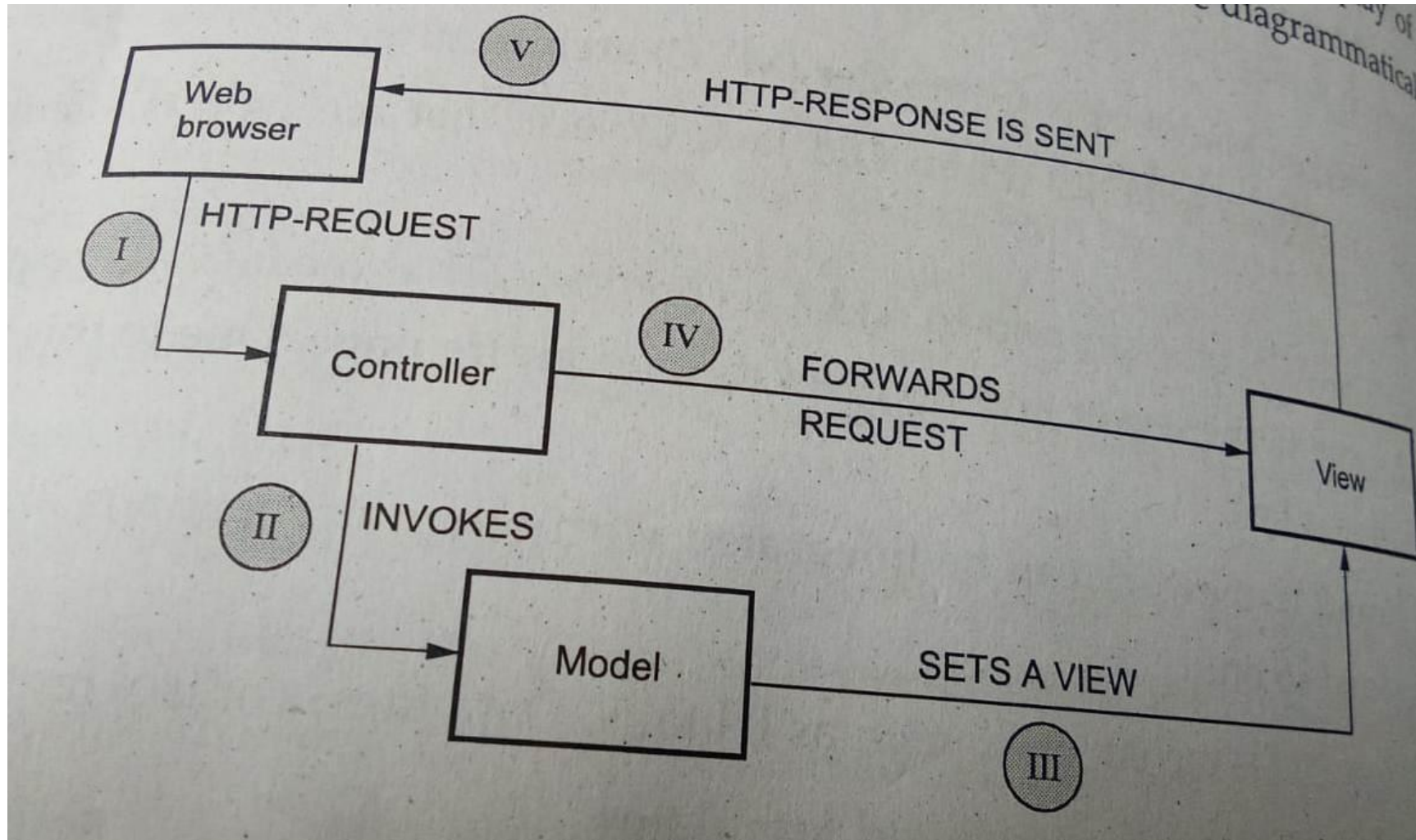
Struts Architecture

1. Normally when client wants any web page he demands it using get or post request or by clicking submit button, then a controller is invoked
2. A controller is nothing but a java class in which business logic is written. The job of controller is to take user input and pass it to the model. The controller in turn invokes the model.
3. Model stores the user's data may be in databases or so, and return some result to the controller.
4. Using this result, the controller figures out user requirements and accordingly forwards the result to the view page.
5. Finally required view can be obtained as a response (i.e. a display of required web page is obtained) on the web browser.

Struts Architecture



Struts Architecture



Struts Architecture

- **Advantage of Model 2 (MVC) Architecture**

- Navigation control is centralized Now only controller contains the logic to determine the next page.
- Easy to maintain
- Easy to extend
- Easy to test
- Better separation of concerns

- **Disadvantage of Model 2 (MVC) Architecture**

- We need to write the controller code self. If we change the controller code, we need to recompile the class and redeploy the application.

Struts 2 - Hello World Example

Create Following Four Components For Any Struts 2 Project

SN.	Components & Description
1	Action Create an action class which will contain complete business logic and control the interaction between the user, the model, and the view.
2	Interceptors Create interceptors if required, or use existing interceptors. This is part of Controller .
3	View Create a JSPs to interact with the user to take input and to present the final messages.
4	Configuration Files Create configuration files to couple the Action, View and Controllers. These files are struts.xml, web.xml, struts.properties .

To create a “Hello World” example, you need to do four things:

- Create a class to store the welcome message (the model)
- Create a server page to present the message (the view)
- Create an Action class to control the interaction between the user, the model, and the view (the controller)
- Create a mapping (struts.xml) to couple the Action class and view

Execution of first Struts Applications

- 1. Environment – Before execution of any struts programs . Following software must be installed.
1.JDK 2.Apache Tomcat 3.Eclipse IDE 4.Strut 2 library – for installing struts 2 libraries get them downloaded from <http://struts.apache.org/download.cgi>
step 1 : Start Eclipse File > New > Dynamic Web Project and enter project name as **HelloWorldStruts2**
- 2. Select all the default options in the next screens and finally check **Generate Web.xml deployment descriptor** option.
- 3. Copy following **Strut 2 jar files** in the lib folder of your project.
 - commons-fileupload-x.y.z.jar
 - commons-io-x.y.z.jar
 - commons-lang-x.y.jar
 - commons-logging-x.y.z.jar
 - commons-logging-api-x.y.jar
 - freemarker-x.y.z.jar
 - javassist-xy.z.GA
 - ognl-x.y.z.jar
 - struts2-core-x.y.z.jar
 - xwork-core.x.y.z.jar

Struts 2 - Hello World Example

4. **Create Action Class-** We need an Action class to act as the Controller. The Action class responds to a user action create a java file HelloWorldAction.java under **Java Resources > src** with a package .**The action class get executed when user clicks the URL., based on it, the JSP page will be displayed.**

```
public class HelloWorldAction {  
    private String name;  
  
    public String execute() throws Exception {  
        return "success";  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

- The Struts 2 framework will create an object of the HelloWorldAction class and call the execute method in response to a user's action (clicking on a hyperlink that sends a specific URL to the Servlet container).
- In this example, the execute method creates an object of class MessageStore and then returns the String constant SUCCESS.
- Note also the public getter method for the private MessageStore object. Since we want to make the MessageStore object available to the view page, HelloWorld.jsp, we need to follow the [JavaBean-style](#) of providing getter and setter methods where needed

Struts 2 - Hello World Example

5. **Create a View- Now we will create a view page.** create the below jsp file **HelloWorld.jsp** in the WebContent folder in your eclipse project. The taglib directive tells us the servlet container that this page is using Struts 2 tag and tags will be preceded by s. The `<s:property>` tag displays the value returned by calling the method `getName` of the `HelloWorldAction`

```
<%@ page contentType = "text/html; charset = UTF-8" %>
<%@ taglib prefix = "s" uri = "/struts-tags" %>

<html>
  <head>
    <title>Hello World</title>
  </head>

  <body>
    Hello World, <s:property value = "name"/>
  </body>
</html>
```

Struts 2 - Hello World Example

6. **Create Main Page**- We also need to create **index.jsp** in the WebContent folder. This file will serve as the initial action URL and calls the method defined in the HelloWorldAction class, accordingly the display of HelloWorld.jsp page will be updated.

- Let's add an Action URL inside index.jsp (see src/main/webapp folder) so the user can click on a link to tell the Struts 2 framework to run the execute method of the HelloWorldAction class and render the HelloWorld.jsp view.
- First add the taglib directive at the top of the jsp `<%@ taglib prefix="s" uri="/struts-tags"%>`. Next add this p tag `<p><a href="<s:url action='hello'/">">Hello World</a></p>` after the h1 tag.
- The Struts url tag creates the URL with an action of hello. The hello action was mapped to the HelloWorldAction class and its execute method. When the user clicks on the above URL it will cause the Struts 2 framework to run the execute method of the HelloWorldAction class. After that method returns the String success (constant SUCCESS), the view page HelloWorld.jsp will be rendered

```
<%@ page language = "java" contentType = "text/html; charset = ISO-8859-1"
    pageEncoding = "ISO-8859-1"%>
<%@ taglib prefix = "s" uri = "/struts-tags"%>
    <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">

<html>
    <head>
        <title>Hello World</title>
    </head>

    <body>
        <h1>Hello World From Struts2</h1>
        <form action = "hello">
            <label for = "name">Please enter your name</label><br/>
            <input type = "text" name = "name"/>
            <input type = "submit" value = "Say Hello"/>
        </form>
    </body>
</html>
```

Struts 2 - Hello World Example

7. Configuration Files- We need a mapping to tie the URL, the HelloWorldAction class (Model), and the HelloWorld.jsp (the view) together. Hence, create struts.xml file under the WebContent/WEB-INF/classes folder. Eclipse does not create the "classes" folder by default, so you need to do this yourself. Place the action node (action name="hello") between the opening and closing package node.

```
<?xml version = "1.0" Encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
    "http://struts.apache.org/dtds/struts-2.0.dtd">

<struts>
    <constant name = "struts.devMode" value = "true" />

    <package name = "helloworld" extends = "struts-default">
        <action name = "hello"
            class = "com.tutorialspoint.struts2.HelloWorldAction"
            method = "execute">
            <result name = "success">/HelloWorld.jsp</result>
        </action>
    </package>
</struts>
```


Struts 2 - Hello World Example

Next step is to create a **web.xml** file which is an entry point for any request to Struts 2. The entry point of Struts application will be a filter defined in deployment descriptor (web.xml). Hence, we will define an entry of `org.apache.struts2.dispatcher.FilterDispatcher` class in web.xml

```
<display-name>Struts 2</display-name>

<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>

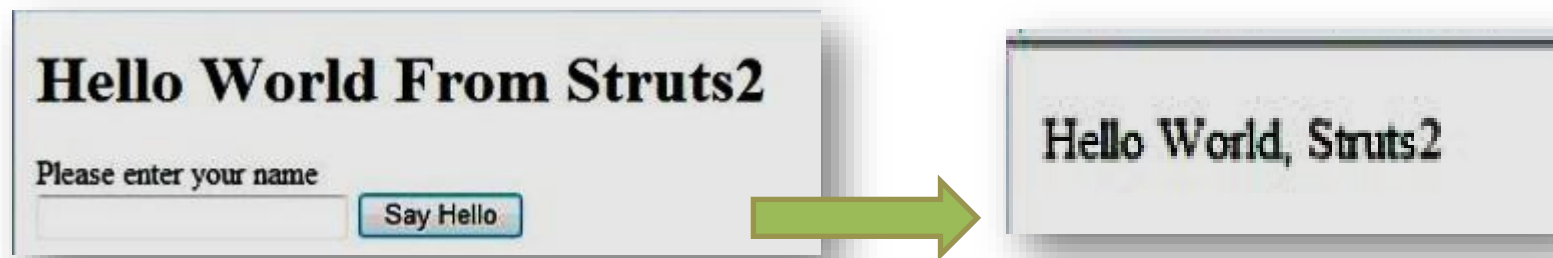
<filter>
  <filter-name>struts2</filter-name>
  <filter-class>
    org.apache.struts2.dispatcher.FilterDispatcher
  </filter-class>
</filter>

<filter-mapping>
  <filter-name>struts2</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
</web-app>
```

Struts 2 - Hello World Example

- **Procedure for Executing the Application**

- Right click on the project name and click **Export > WAR File** to create a War file.
- Then deploy this WAR in the Tomcat's webapps directory.
- Finally, start Tomcat server and try to access URL **<http://localhost:8080/HelloWorldStruts2/index.jsp>**. This will give you following screen –
- Enter a value "Struts2" and submit the page. You should see the next page



Outline

Struts: Overview,

architecture,

configuration,

actions,

interceptors,

result types,

validations,

localization,

exception handling,

annotations.

Struts Configuration

- There are two main configuration files used in struts and those are struts.xml and web.xml
- Struts.xml – the struts.xml file is used to initialize the resources such as interceptors, Actions Classes.
- The first thing to note is the **DOCTYPE**. All struts configuration file needs to have the correct doctype as shown in our little example. <struts> is the root tag element, under which we declare different packages using <package> tags. Here <package> allows separation and modularization of the configuration. This is very useful when you have a large project and project is divided into different modules.
- For example, if your project has three domains - business_application, customer_application and staff_application, then you could create three packages and store associated actions in the appropriate package.

The package tag has the following attributes –

Sr.No	Attribute & Description
1	name (required) The unique identifier for the package
2	extends Which package does this package extend from? By default, we use struts-default as the base package.
3	abstract If marked true, the package is not available for end user consumption.
4	namespace Unique namespace for the actions

- The **constant** tag along with name and value attributes should be used to override any of the following properties defined in **default.properties**, like we just set **struts.devMode** property. Setting **struts.devMode** property allows us to see more debug messages in the log file.
- We define **action** tags corresponds to every URL we want to access and we define a class with execute() method which will be accessed whenever we will access corresponding URL.
- Results determine what gets returned to the browser after an action is executed. The string returned from the action should be the name of a result. Results are configured per-action as above, or as a "global" result, available to every action in a package. Results have optional **name** and **type** attributes. The default name value is "success".

```
<?xml version = "1.0" Encoding = "UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
    "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
    <constant name = "struts.devMode" value = "true" />
    <package name = "helloworld" extends = "struts-default">

        <action name = "hello"
            class = "com.tutorialspoint.struts2.HelloWorldAction"
            method = "execute">
            <result name = "success">/HelloWorld.jsp</result>
        </action>

        <-- more actions can be listed here -->

    </package>
    <-- more packages can be listed here -->

</struts>
```

Struts Configuration

- **The Struts-config.xml File**

- The struts-config.xml configuration file is a link between the View and Model components in the Web Client.
- The web.xml is a configuration file which determines how elements for the HTTP request are processed by the servlet container.
- It is not a strictly a struct2 configuration file, but it is a file that needs to be configured for struct2 work
- The web.xml file needs to be created under the folder WebContent/WEB-INF
- This file contains the <welcome-file-list> tag using which the first file to be loaded on execution of struts project is specified. For example :-


```
<welcome-file-list>
```

```
<welcome-file>index.jsp</welcome-file>
```

```
</welcome-file-list>
```

- The simple meaning of above code is that as soon as you execute your struts application , load the index.jsp file

```
<filter-mapping>
```

```
<filter-name>struts2</filter-name>
```

```
<url-pattern>/*</url-pattern>
```

```
</filter-mapping>
```

- The above code indicated that <url - pattern> is /* means that all urls will be parsed by the struts filter.

Struts 2 - Actions

- The actions are basically the java programs that contain the processing logic and satisfy the requests made by the user.
- Each URL is mapped to these actions.
- Actions performs two tasks –
- action also serves in two other important capacities - .
 - Firstly, the action plays an important role in the transfer of data from the request through to the view, whether its a JSP or other type of result.
 - Secondly, the action must assist the framework in determining which result should render the view that will be returned in the response to the request.

- Action class is a POJO (plain old java object) class which has attributes and method. There must be one method that should always be present in action class and that is – execute() method.
- By default execute() method is called when client request goes to action class.
- This class is bean class . That means it is a java class containing getxx and setxx methods.

Struts 2 - Actions

Create Action

- The only requirement for actions in **Struts2** is that there must be one no argument .
- If the no-argument method is not specified, the default is to use the execute() method.
- Optionally you can extend the **ActionSupport** class which implements six interfaces including **Action** interface. The Action interface is as follows –

```
public interface Action {  
    public static final String SUCCESS = "success";  
    public static final String NONE = "none";  
    public static final String ERROR = "error";  
    public static final String INPUT = "input";  
    public static final String LOGIN = "login";  
    public String execute() throws Exception;  
}
```

Action method in the Hello World example –

```
public class HelloWorldAction extends ActionSupport {  
    private String name;  
  
    public String execute() throws Exception {  
        if ("SECRET".equals(name)) {  
            return SUCCESS;  
        } else {  
            return ERROR;  
        }  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Struts 2 - Interceptors

- Interceptors is an object which intercepts an action dynamically . It executed before and after the action execution.
- In struts 2 , interceptor is used to perform operations such as validation, exception handling, internalization, displaying intermediate result etc.
- Following is a list of interceptors that are preconfigured and can be ready to use

Struts 2 - Interceptors

Interceptor & Description

Alias-Allows parameters to have different name aliases across requests.

Checkbox- Assists in managing checkboxes by adding a parameter value of false for checkboxes that are not checked.

Locale-Keeps track of the selected locale during a user's session.

Exception- Maps exceptions that are thrown from an action to a result, allowing automatic exception handling via redirection.

fileUpload- Facilitates easy file uploading.

Params- Sets the request parameters on the action.

Prepare- This is typically used to do pre-processing work, such as setup database connections.

Timer- Provides simple profiling information in the form of how long the action takes to execute.

Token-Checks the action for a valid token to prevent duplicate form submission.

- We can create our own inceptor object. This custom based inceptor can be plugged in struts.
- Framework creates an object of ActionInvocation that encapsulates the action and all the interceptors configured for that action
- Each interceptors are called before the action gets called.
- Once the action is called and result is generated, each interceptors are again called in reverse order to perform post processing work

Struts 2 - Interceptors

- **Create Interceptors**
- We will use the **timer** interceptor whose purpose is to measure how long it took to execute an action method.
- At the same time, using **params** interceptor whose purpose is to send the request parameters to the action.
- modify the **struts.xml** file to add an interceptor

```
<interceptor-ref name = "params"/>  
<interceptor-ref name = "timer" />
```


Struts 2 - Interceptors

- **Create Custom Interceptors**
- Using custom interceptors in your application is an elegant way to provide crosscutting application features.
- `init()` method provides a way to initialize the interceptor, and the `destroy()` method provides a facility for interceptor cleanup.
- The **ActionInvocation** object provides access to the runtime environment.

```
public interface Interceptor extends Serializable {  
    void destroy();  
    void init();  
    String intercept(ActionInvocation invocation)  
        throws Exception;  
}
```

Struts 2 - Results & Result Types

- **<results>** tag plays the role of a **view** in the Struts2 MVC framework.
- The action is responsible for executing the business logic.
- The next step after executing the business logic is to display the view using the **<results>** tag
- After executing the action class, the target view page will be executed based on configuration in struts.xml
- The **<result>** tag is used in struts.xml to render the view page. This result tag is provided with predefined result types such as dispatcher, redirect and so on.
- For example **<result type = “success” type = “redirect” >**

Struts 2 - Results & Result Types

Struts comes with a number of predefined **result types**

1. **Dispatcher**
2. **Velocity,**
3. **Freemaker,**
4. **Redirect**
5. **XSLT and Tiles.**

Struts 2 - Results & Result Types

- **The Dispatcher Result Type**

- The **dispatcher** result type is the default type, and is used if no other result type is specified. It's used to forward to a servlet, JSP, HTML page, and so on, on the server. It uses the *RequestDispatcher.forward()* method.
 - `<result name = "success" type = "dispatcher">`
 - `/HelloWorld.jsp`
 - `</result>`

- **The FreeMaker Result Type**

- Freemaker is a popular templating engine that is used to generate output using predefined templates.
- Let us now create a Freemaker template file called **hello.fm** with the following contents –
 - **Hello World \${name}**
- The above file is a template where **name** is a parameter which will be passed from outside using the defined action.

Struts 2 - Results & Result Types

- **The Redirect Result Type**

- The **redirect** result type calls the standard *response.sendRedirect()* method, causing the browser to create a new request to the given location.
- We can provide the location either in the body of the <result...> element or as a <param name = "location"> element. Redirect also supports the **parse** parameter. Here's an example configured using XML –
 - <result name = "success" type = "redirect">
 - <param name = "location">
 - /NewWorld.jsp
 - </param >
 - </result>

Struts 2 - Validations Framework

At the Struts core, we have the validation framework that assists the application to run the rules to perform validation before the action method is executed.

we will take an example of an **Employee** whose age ,we will put validation to make sure that the user always enters a age in a range between 28 and 65.

You can validate data in action class also through validate() method

If the validation fails an error is added using the addFieldError() method

Struts 2 - Validations Framework

Create Main Page- JSP file **index.jsp**, which will be used to collect Employee related information mentioned above.

```
<html>
  <body>
    <s:form action = "empinfo" method = "post">
      <s:textfield name = "age" label = "Age" size = "20" />
      <s:submit name = "submit" label = "Submit" align="center" />
    </s:form>
  </body>
</html>
```

Struts 2 - Validations Framework

Create Views- We will use JSP file success.jsp which will be invoked in case defined action returns SUCCESS.

```
<html>  
  <body>  
    Employee Information is captured successfully.  
  </body>  
</html>
```


Struts 2 - Validations Framework

Create Action- action class **Employee** with a method called **validate()** in **Employee.java** file. So let us define a small action class **Employee**, and then add a method called **validate()** as shown below in **Employee.java** file. Make sure that your action class extends the **ActionSupport** class, otherwise your validate method will not be executed

```
public class Employee extends ActionSupport {  
    private int age;  
    public String execute() {return SUCCESS; }  
    public int getAge() { return age; }  
    public void setAge(int age) { this.age = age; }  
    if (age < 28 || age > 65) {  
        addFieldError("age","Age must be in between 28 and 65"); }  
}
```

Struts2 - Localization

- Internationalization (i18n) is the process of planning and implementing products and services so that they can easily be adapted to specific local languages and cultures, a process called localization.
- The internationalization process is a process of translation or localization.
- Internationalization is abbreviated **i18n** because the word starts with the letter “i” and ends with “n”, and there are 18 characters between the first i and the last n.
- Struts2 provides localization, i.e., internationalization (i18n) support through **resource bundles, interceptors and tag libraries** in the following places –
 - The UI Tags
 - Messages and Errors.
 - Within action classes.

Struts2 – Localization

Concept of Resource Bundles

- Struts2 uses resource bundles to provide multiple language and locale options to the users of the web application.
- You don't need to worry about writing pages in different languages.
- All you have to do is to create a resource bundle for each language that you want.
- The resource bundles will contain titles, messages, and other text in the language of your user.
- Resource bundles are the file that contains the key/value pairs for the default language of your application.
- The simplest naming format for a resource file is –
 - `bundlename_language_country.properties`
- Here, bundlename could be ActionClass, Interface, SuperClass, Model, Package, Global resource properties.
- Next part language_country represents the country locale for example,
 - English (United States) locale is represented by `en_US` etc.

Struts2 – Localization

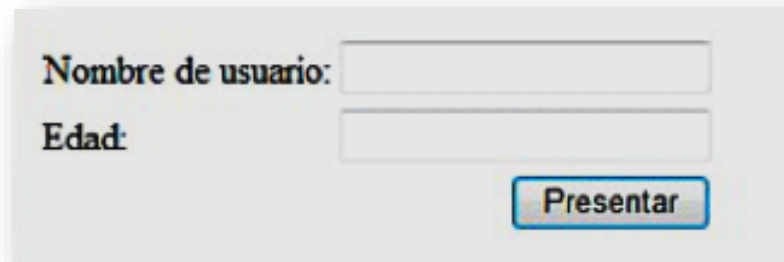
- To develop your application in multiple languages, you should maintain multiple property files corresponding to those languages/locale and define all the content in terms of key/value pairs.
- For example, if you are going to develop your application for US English (Default), Spanish, and French, then you would have to create three properties files.
 - **global.properties** – By default English (United States) will be applied
 - **global_fr.properties** – This will be used for French locale.
 - **global_es.properties** – This will be used for Spanish locale.

Struts2 – Localization **Example**



A screenshot of a web form titled "Select Locale". At the top, there are three underlined links: "English", "Spanish", and "France". Below these links are two input fields: "Name:" and "Age:". At the bottom right of the form is a "Submit" button.

Now select any of the languages, let us say we select **Spanish**, it would display the following result –



A screenshot of a web form in Spanish. It contains two input fields: "Nombre de usuario:" and "Edad:". At the bottom right is a "Presentar" button.

Struts 2 - Exception Handling

- **Struts** provides an easier way to handle uncaught exception and redirect users to a dedicated error page.
- You can easily configure Struts to have different error pages for different exceptions.
- Struts makes the exception handling easy by the use of the "exception" interceptor.

- Create a file called **Error.jsp**

```
<html>
```

```
  <body>
```

```
    This is my custom error page
```

```
  </body>
```

```
</html>
```

- Let us now configure Struts to use this error page in case of an exception.
- Let us modify the **struts.xml** by adding following line–
<result name = "error"> /Error.jsp </result>

Struts 2 - Annotations

- Java annotation is a tag used to represent the metadata. The metadata is a data about classes , methods or fields.
- Annotations in java are used to provide some addition information. For instance- if user is entering his age in the text fields, then that age must not be negative or it should not be blank- such type of information can be provided to the user using annotations
- A java annotation in its shortest form looks like this : @Entity
- Commonly , we can use 3 annotations :
 1. @Action annotation is used to mark the action class.
 2. @Results annotation is used to define multiple results for one action
 3. @Result annotation is used to display single result

- To support the annotation we require following files in the java program to be imported as –
- `Import org.apache.struts2.convention.annotation.Action;`
- `Import org.apache.struts2.convention.annotation.Result;`
- `Import org.apache.struts2.convention.annotation.Results;`
- `Import com.opensymphony.xwork2.validator.annotation;`

Struts 2 - Annotations

To start using annotations in your project, make sure you have included the following jar files in your **WebContent/WEB-INF/lib** folder

struts2-convention-plugin-x.y.z.jar

asm-x.y.jar

antlr-x.y.z.jar

commons-fileupload-x.y.z.jar

commons-io-x.y.z.jar

commons-lang-x.y.jar

commons-logging-x.y.z.jar

commons-logging-api-x.y.jar

freemarker-x.y.z.jar

javassist-xy.z.GA

ognl-x.y.z.jar

struts2-core-x.y.z.jar

xwork-core.x.y.z.jar

Struts 2 – Annotations **Types**

Annotations Types

Namespace Annotation (Action Annotation)

Result Annotation - (Action Annotation)

Results Annotation - (Action Annotation)

The @Results annotation defines a set of results for an Action.

After Annotation - (Interceptor Annotation)

The @After annotation marks a action method that needs to be called after the main action method and the result was executed. Return value is ignored.

Before Annotation - (Interceptor Annotation)

The @Before annotation marks a action method that needs to be called before the main action method and the result was executed. Return value is ignored.

EmailValidator Annotation - (Validation Annotation)

IntRangeFieldValidator Annotation - (Validation Annotation)

RequiredFieldValidator Annotation - (Validation Annotation)

Struts 2 – Annotations Example

Validation on Name field as required and Age as range between 28 and 65.

```
public class Employee extends ActionSupport {
```

```
    private String name;
```

```
    private int age;
```

```
@RequiredFieldValidator( message = "The name is required")
```

```
    public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }
```

```
@IntRangeFieldValidator(message = "Age must be in between 28 and 65", min = "28", max = "65")
```

```
    public int getAge() { return age; }
```

```
    public void setAge(int age) { this.age = age; } }
```